

Programozás Alapjai

Dr. Gergely Tamás
Dr. Kicsi András

Szegedi Tudományegyetem
Informatikai Intézet
Szoftverfejlesztés Tanszék

2025

(v1207)



Tartalomjegyzék

- 
- 1 Bemutatók**
 - Kurzus információk
 - Infrastruktúra szabályzatok
 - Mentorálás
 - 2 Python alapok**
 - Bevezető
 - Pénzváltás (1. verzió)
 - Pénzváltás (2. verzió)
 - 3 Alapfogalmak**
 - Alapfogalmak
 - Alapvető programelemek fogalmak
 - 4 Python feltételes ciklusok**
 - Röppálya számítás
 - Röppálya szimuláció
 - 5 Szoftverfejlesztési életciklus modellek**
 - A programozás fázisai
 - 6 Python sztringek**
 - Python sztringek
 - Basename meghatározása
 - 7 Vezérlési szerkezetek**
 - Bevezetés
 - Szekvenciális vezérlés
 - Függvények
 - 8 Python listák**
 - Python list és tuple adattípusok
 - Python listáshoz tartozó műveletek
 - Basename és eltelt idő
 - 9 Vezérlési szerkezetek**
 - Szelekciós vezérlések
 - 10 Python iterációk**
 - Csúszóátlag számítása
 - 11 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 12 Python halmazok, comprehension**
 - Python set és frozenset adattípusok
 - Tér legtovábbi pontjai
 - 13 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 14 Python dictionary, függvény mint típus**
 - Python dict adattípus
 - Python comprehension
 - Python function adattípus
 - 15 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 16 Python fájlkezelés**
 - Fájlkezelés
 - 17 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 18 Python formátált IO**
 - Python formátált IO
 - 19 Adatkezelés, adattárolás**
 - Az adatkezelés szintjei
 - Az adattárolás jellemzői
 - 20 Python típusok**
 - Python numerikus típusok
 - 21 Absztrakt adattípusok**
 - Egyszerű adattípusok
 - Összetett adattípusok
 - 22 Python típusok**
 - További Python típusok
 - Python function adattípus
 - 23 Absztrakt adattípusok**
 - Összetett adattípusok
 - 24 Tesztelés**
 - Tesztelés
 - Tesztelés Python-ban
 - A pytest
 - 25 Vizualizáció és egyéb modulok**
 - Fájlkezelő modulok
 - Vizualizáció
 - 26 More Python**
 - A Python program végrehajtásának folyamata
 - Függvényparaméterek és argumentumok
 - Python unpacking
 - Kivételkezelés

Tartalomjegyzék

- 
- 1 Bemutatók**
 - Kurzus információk
 - Infrastruktúra szabályzatok
 - Mentorálás
 - 2 Python alapok**
 - Bevezető
 - Pénzváltás (1. verzió)
 - Pénzváltás (2. verzió)
 - 3 Alapfogalmak**
 - Alapfogalmak
 - Alapvető programelemek fogalmak
 - 4 Python feltételes ciklusok**
 - Röppálya számítás
 - Röppálya szimuláció
 - 5 Szoftverfejlesztési életciklus modellek**
 - A programozás fázisai
 - 6 Python sztringek**
 - Python sztringek
 - Basename meghatározása
 - 7 Vezérlési szerkezetek**
 - Bevezetés
 - Szekvenciális vezérlés
 - Függvények
 - 8 Python listák**
 - Python list és tuple adattípusok
 - Python listáshoz adattípusok műveletei
 - Basename és eltelt idő
 - 9 Vezérlési szerkezetek**
 - Szelekciós vezérlések
 - 10 Python iterációk**
 - Csúszóátlag számítása
 - 11 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 12 Python halmazok, comprehension**
 - Python set és frozenset adattípusok
 - Tér legtovábbi pontjai
 - 13 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 14 Python dictionary, függvény mint típus**
 - Python dict adattípus
 - Python comprehension
 - Python function adattípus
 - 15 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 16 Python fájlkezelés**
 - Fájlkezelés
 - 17 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 18 Python formátált IO**
 - Python formátált IO
 - 19 Adatkezelés, adattárolás**
 - Az adatkezelés szintjei
 - Az adattárolás jellemzői
 - 20 Python típusok**
 - Python numerikus típusok
 - 21 Absztrakt adattípusok**
 - Egyszerű adattípusok
 - Összetett adattípusok
 - 22 Python típusok**
 - További Python típusok
 - Python function adattípus
 - 23 Absztrakt adattípusok**
 - Összetett adattípusok
 - 24 Tesztelés**
 - Tesztelés
 - Tesztelés Python-ban
 - A pytest
 - 25 Vizualizáció és egyéb modulok**
 - Fájlkezelő modulok
 - Vizualizáció
 - 26 More Python**
 - A Python program végrehajtásának folyamata
 - Függvényparaméterek és argumentumok
 - Python unpacking
 - Kivételkezelés

A *string* adattípus

ABSZTRAKT adattípus

- Értékkészlete

- $\mathbb{U}^* = \{c_0c_1 \dots c_{n-1} \mid \forall i : 0 \leq i < n, c_i \in \mathbb{U}; n \in \mathbb{N}_0\}$, ahol $\mathbb{U}$ egy tetszőleges szimbólumkészlet.
- Informatikában manapság az $\mathbb{U}$ a *Unicode* karakterkészletet [9] jelenti.
- A *string* absztrakt adattípus tulajdonképpen egy lista.

- Műveletei (alpműveletek)

- karakter elérése (kiolvasása/módosítása)
- összehasonlítás (= és <, ≤, >, ≥ lexikografikus rendezés szerint)
- hossz meghatározása
- karakter törlése, beszúrása
- összefűzés, darabolás
- részsstring

Az `str` (*string*) adattípus

Python VIRTUÁLIS adattípus

- Az absztrakt *string* adattípus megvalósítása Python-ban.
- Az értékkészlete *Unicode* karakterek (elméletben) tetszőleges hosszú listáiból áll.
- Python-ban nincs karakter típus, csak 1 hosszúságú *string*.
- Az `str` *immutable* típus: ha egyszer létrehoztunk egy *string*et, nem tudjuk megváltoztatni.



A *halmaz* adattípus

ABSZTRAKT adattípus

- Értékkészlete
 - $P(\mathbb{T}) = \{S \mid S \subseteq \mathbb{T}\}$, ahol $\mathbb{T}$ egy tetszőleges típus.
- Műveletei (alapműveletek)
 - törlés, beszúrás, elemszám
 - unió, metszet, különbség, komplementer
 - egyenlőség reláció, részhalmaz, tartalmazás



A set (*halmaz*) adattípus

Python VIRTUÁLIS adattípus

- A *halmazok* egyedi elemek rendezetlen összességei.
- A `set` *mutable* típus, de csak *immutable* elemeket tartalmazhat.

```
>>> primes = {2,7,5,2,3,5}
>>> primes.add(3)
>>> primes
{2, 3, 5, 7}
>>> primes.add(11)
>>> primes
{2, 3, 5, 7, 11}
>>> primes.add(('Péntek', 13))
>>> primes
{2, 3, 5, 7, 11, ('Péntek', 13)}
>>> primes.add(['list'])
Traceback (most recent call last):
File "<stdin>", line 1, in <module>
TypeError: unhashable type: 'list'
```

A frozenset (*halmaz*) adattípus

Python VIRTUÁLIS adattípus

- A *halmazok* egyedi elemek rendezetlen összességei.
- A `frozenset` *immutable* típus, és csak *immutable* elemeket tartalmazhat.
- A `frozenset` típus tulajdonképpen a `set` típus *immutable* verziója.
- A `frozenset` ennek megfelelően nem rendelkezik a `set` olyan műveleteivel, amelyek megváltoztathatnák az értékét.

```
>>> primes = frozenset({2,7,5,2,3,5})
>>> primes.add(3)
Traceback (most recent call last):
File "<stdin>", line 1, in <module>
AttributeError: 'frozenset' object has no attribute 'add'
```

Klikkek meghatározása

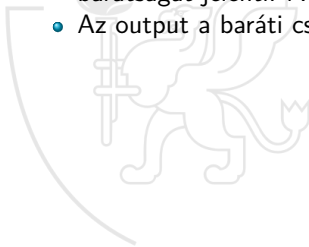
Problémafelvetés és specifikáció

- Problémafelvetés

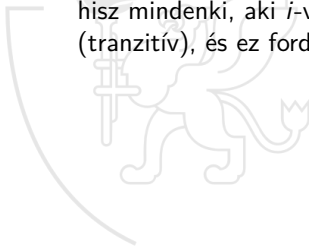
- Egy közösségben az emberek közötti barátsági kapcsolat alapján meg kell határoznunk a klikkeket!

- Specifikáció

- Az emberek számát (N) előre bekérjük, és az embereket az $1, \dots, N$ számokkal jelöljük.
 - A program inputja i, j számpárok sorozata ($1 \leq i, j \leq N$), ami az i . és j . sorszámú ember barátságát jelenti. A sorozat (és az input) végét egy üres sor jelöli.
 - Az output a baráti csoportok tagjainak felsorolása.



- A matematika nyelvén:
 - Tegyük fel, hogy a barátság reflexív, szimmetrikus és tranzitív reláció. Meg kell határozni az R relációt tartalmazó legszűkebb ekvivalencia reláció szerinti osztályozást.
 - Induljunk ki abból, hogy minden személy csak saját magával van barátságban (reflexív); tehát képezzük az i egyelemű halmazokat.
 - Ha az input számpárokat valameddig feldolgozva meghatároztuk az osztályozást, a következő (i, j) számpárt beolvassa össze kell vonni azt a két részhalmazt, amelybe i illetve j tartozik, hisz mindenki, aki i -vel barátságban van, az barátságban van j minden barátjával is (tranzitív), és ez fordítva is igaz (szimmetrikus).



Klikkek meghatározása [1/2]

klikk.py [1–13]

```
1#!/usr/bin/env python3
2"""
3Programozás, Alapjai, előadás, példaprogram
4SZTE, TTK, Informatikai Intézet, Szoftverfejlesztés, Tanszék
5Gergely, Tamás, gertom@inf.u-szeged.hu, 2024-05-31
6cc-by-nc-nd
7
8A beolvasott R relációt tartalmazó, legszűkebb ekvivalencia
9reláció szerinti osztályozást határozzuk meg.
10"""
11def main():
12    n = int(input("Hány ember van? "))
13    klikkek = [set([i]) for i in range(1, n+1)]
```

Klikkek meghatározása [2/2]

klikk.py [15–32]

```
15 while (r := input("Kik ismerik egymást (a,b)?")):
16     (a, b) = [int(i) for i in r.split(',')]
17     for ai in range(0, len(klikkek)):
18         if a in klikkek[ai]:
19             break
20     for bi in range(0, len(klikkek)):
21         if b in klikkek[bi]:
22             break
23     if ai != bi:
24         klikkek[ai].update(klikkek[bi])
25         del klikkek[bi]
26
27 print("A baráti körök:")
28 for k in klikkek:
29     print(k)
30
31 if __name__ == "__main__":
32     main()
```

Prímszámok meghatározása

Problémafelvetés és specifikáció

- Problémafelvetés
 - Határozzuk meg az adott N természetes számnál kisebb prímszámokat.
- Specifikáció
 - Legyen N a program parancssori argumentuma, egy pozitív egész szám.
 - A kimenet a $[0, N)$ intervallumba eső prímszámok sorozata.



Prímszámok meghatározása

Algoritmustervezés

- Első ötletünk egy „favágó” algoritmus lehet: minden számról külön-külön eldöntjük, hogy prím-e vagy sem.
- Hogyan lehet eldönteni egy n számról, hogy prím-e?
 - Akkor prím, ha pontosan két egész osztója van, 1 és n , és $n \neq 1$.
 - Vagyis, ha $n > 1$ és a $[2, n - 1]$ intervallumban nincs egész osztója. (A $[2, 1]$ intervallum üres!)
 - Valójában elég a $[2, \lfloor n/2 \rfloor]$, sőt, a $[2, \lfloor \sqrt{n} \rfloor]$ intervallumot vizsgálni.
- Ez az algoritmus Python nyelven igen sokféleképpen megvalósítható.

Prím – lassú algoritmus – for ciklussal

prim_slow_for.py [11–35]

```
11 from sys import argv
12
13 def prim(p):
14     if p == 1:
15         return False
16     d = 2
17     for d in range(2, (p // 2) + 1):
18         if (p % d) == 0:
19             return False
20     return True
21
22 def printprimes(l, limit):
23     print(f"A prímszámok {limit}-ig:")
24     db = 0
25     for p in l:
26         db += 1
27         print("%9d" % (p), end = '' if db % 8 != 0 else '\n')
28     print()
29
30 def main():
31     limit = int(argv[1])
32     printprimes([p for p in range(2, limit) if prim(p)], limit)
33
34 if __name__ == "__main__":
35     main()
```

Prím – lassú algoritmus – while ciklussal

prim_slow_while.py [11–36]

```
11 from sys import argv
12
13 def prim(p):
14     if p == 1:
15         return False
16     d = 2
17     while d * d <= p:
18         if (p % d) == 0:
19             return False
20         d += 1
21     return True
22
23 def printprimes(l, limit):
24     print(f"A prím számok {limit}-ig:")
25     db = 0
26     for p in l:
27         db += 1
28         print("%9d" % (p), end = '' if db % 8 != 0 else '\n')
29     print()
30
31 def main():
32     limit = int(argv[1])
33     printprimes([p for p in range(2, limit) if prim(p)], limit)
34
35 if __name__ == "__main__":
36     main()
```

Prím – lassú algoritmus – while ciklus, direkt kiíratás

prim_slow_range.py [11–34]

```
11 from sys import argv
12
13 def prim(p):
14     if p == 1:
15         return False
16     d = 2
17     while d * d <= p:
18         if (p % d) == 0:
19             return False
20         d += 1
21     return True
22
23 def main():
24     limit = int(argv[1])
25     print(f"A prím számok {limit}-ig:")
26     db = 0
27     for p in range(2, limit):
28         if prim(p):
29             db += 1
30             print("%9d" % (p), end = ' ' if db % 8 != 0 else '\n')
31     print()
32
33 if __name__ == "__main__":
34     main()
```

Prím – lassú algoritmus – az all() függvény segítségével

prim_slow_all.py [11–29]

```
11 from sys import argv
12
13 def prim(p):
14     return all(p % d != 0 for d in range(2, p)) if p > 1 else False
15
16 def printprimes(l, limit):
17     print(f"A prím számok {limit}-ig:")
18     db = 0
19     for p in l:
20         db += 1
21         print("%9d" % (p), end = '' if db % 8 != 0 else '\n')
22     print()
23
24 def main():
25     limit = int(argv[1])
26     printprimes([p for p in range(2, limit) if prim(p)], limit)
27
28 if __name__ == "__main__":
29     main()
```

Prím – lassú algoritmus – list comprehension az all() segítségével

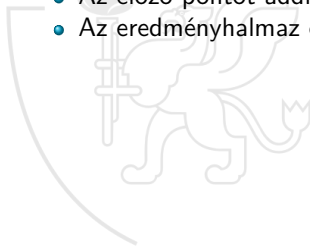
prim_slow_lc.py [11–26]

```
11 from sys import argv
12
13 def printprimes(l, limit):
14     print(f"A prím számok {limit}-ig:")
15     db = 0
16     for p in l:
17         db += 1
18         print("%9d" % (p), end = '' if db % 8 != 0 else '\n')
19     print()
20
21 def main():
22     limit = int(argv[1])
23     printprimes([p for p in range(2, limit) if all(p % d != 0 for d in range(2, p))], limit)
24
25 if __name__ == "__main__":
26     main()
```

Prímszámok meghatározása

Algoritmustervezés

- A „minden számról külön-külön eldöntjük, hogy prím-e vagy sem” algoritmus nem igazán hatékony.
- Ezért a jól ismert Erathosztenészi szita algoritmust valósítjuk meg:
 - A halmazt kezdetben feltöltjük az egynél nagyobb páratlan számokkal (és belerakjuk a 2-t is).
 - Megkeressük a halmaz még nem feldolgozott legkisebb elemét (ez prímszám lesz) és töröljük a többszöröseit.
 - Az előző pontot addig ismételjük, míg el nem érjük az N gyökét.
 - Az eredményhalmaz csak a prímszámokat fogja tartalmazni.



Prím – gyors algoritmus – lista és range()

prim_fast_list.py [11–34]

```
11 from sys import argv
12
13 def prim(limit):
14     primes = [2] + [n for n in range(3, limit, 2)]
15     for p in primes:
16         for mp in range(p * p, limit, 2 * p):
17             if mp in primes:
18                 primes.remove(mp)
19     return primes
20
21 def printprimes(l, limit):
22     print(f"A prím számok {limit}-ig:")
23     db = 0
24     for p in l:
25         db += 1
26         print("%9d" % (p), end = '' if db % 8 != 0 else '\n')
27     print()
28
29 def main():
30     limit = int(argv[1])
31     printprimes(prim(limit), limit)
32
33 if __name__ == "__main__":
34     main()
```

Prím – gyors algoritmus – halmaz és range()

prim_fast.py [11–34]

```
11 from sys import argv
12
13 def prim(limit):
14     primes = {2} | {n for n in range(3, limit, 2)}
15     for p in range(3, limit, 2):
16         if p in primes:
17             for pp in range(p * p, limit, 2 * p):
18                 primes.discard(pp)
19     return primes
20
21 def printprimes(l, limit):
22     print(f"A prím számok {limit}-ig:")
23     db = 0
24     for p in l:
25         db += 1
26         print("%9d" % (p), end = '' if db % 8 != 0 else '\n')
27     print()
28
29 def main():
30     limit = int(argv[1])
31     printprimes(prim(limit), limit)
32
33 if __name__ == "__main__":
34     main()
```

Prím – gyors algoritmus – halmaz és while

prim_fast_manual.py [11–38]

```
11 from sys import argv
12
13 def prim(limit):
14     primes = {2} | {n for n in range(3, limit, 2)}
15     p = 3
16     while p * p < limit:
17         if p in primes:
18             pp = p * p
19             while pp < limit:
20                 primes.discard(pp)
21                 pp += 2 * p
22             p += 2
23     return primes
24
25 def printprimes(l, limit):
26     print(f"A prím számok {limit}-ig:")
27     db = 0
28     for p in l:
29         db += 1
30         print("%9d" % (p), end = '' if db % 8 != 0 else '\n')
31     print()
32
33 def main():
34     limit = int(argv[1])
35     printprimes(prim(limit), limit)
36
37 if __name__ == "__main__":
38     main()
```

Prímszámok meghatározása

Teljesítmény

- Hasonlítsuk össze az egyes verziók futásidejét, teljesítményét
 - Az előre bekészített szkript nagyságrendileg megmutatja a különbségeket

```
1$ ./primspeed.sh
```



Az asszociatív tömb adattípus

ABSZTRAKT adattípus

- Értékkészlete

- $D(\mathbb{K}, \mathbb{V}) = \{E \subseteq \mathbb{K} \times \mathbb{V} \mid (k, v) \in E, k \in \mathbb{K}, v \in \mathbb{V}, ((k_1, v_1) \in E \wedge (k_2, v_2) \in E) \rightarrow k_1 \neq k_2\}$, ahol $\mathbb{K}$ és $\mathbb{V}$ tetszőleges típusok (akár $\mathbb{K} = \mathbb{V}$).

- Műveletei (alapműveletek)

- elem elérése (kiolvasása/módosítása)
- összehasonlítás (=)
- méret meghatározása
- törlés, beszúrás



A dict (*dictionary*) adattípus

Python VIRTUÁLIS adattípus

- A *dictionary*-k asszociatív tömbök, kulcs-érték párok *listáját* tárolják.
- Felfoghatók olyan *listáknak*, amiket potenciálisan bármivel (nem csak egész számokkal) indexelhetünk.
- A dict *mutable* típus: az elemeit megváltoztathatjuk, törölhetjük, beszúrhatunk elemeket, stb.

```
>>> d = {'name': 'Garfield'}
>>> d['name']
'Garfield'
>>> d['age'] = 45
>>> d[0] = '1978'
>>> d
{'name': 'Garfield', 'age': 45, 0: '1978'}
```

Égitestek fényessége [1/2]

egitestek.py [1–27]

```
1#!/usr/bin/env python3
2"""
3Programozás, Alapjai, előadás, példaprogram
4SZTE, TTIK, Informatikai Intézet, Szoftverfejlesztés, Tanszék
5Gergely, Tamás, gertom@inf.u-szeged.hu, 2025-10-16
6cc-by-nc-nd
7
8Legfényesebb csillag.
9"""
10from math import inf, log10
11from ter import tav
12
13def beolvas():
14    try:
15        line = input('Név: abszolút, magnitúdó, x, y, z? ')
16        name, line = line.split(':')
17        mag, px, py, pz = map(float, line.split(','))
18        return name.strip(), {"magnitude": mag, "position": (px, py, pz)}
19    except:
20        return None
21
22def am(M, d):
23    """Látszólagos magnitúdó
24    M: abszolút magnitúdó
25    d: távolság parsec-ben
26    """
27    return M + 5 * (log10(d) - 1)
```

Égitestek fényessége [2/2]

egitestek.py [29–54]

```
29 def fenyesség(observer, star):
30     d = tav(observer["position"], star["position"])
31     return am(star["magnitude"], d)
32
33 def legfenyesebb(stars):
34     minmag = inf
35     o, s = None, None
36     for i in stars:
37         for j in stars:
38             if i == j:
39                 continue
40             if (ujmag := fenyesség(stars[i], stars[j])) < minmag:
41                 minmag = ujmag
42                 o, s = i, j
43     return o, s, minmag
44
45 def main():
46     stars = {}
47     while r := beolvas():
48         n, d = r
49         stars[n] = d
50     o, s, m = legfenyesebb(stars)
51     print(f"A legfényesebbnek a(z) {s} csillag látszik a(z) {o} csillag környezetéből {m:.3f} magnitúdóval.")
52
53 if __name__ == "__main__":
54     main()
```

A függvény adattípus

ABSZTRAKT adattípus

- Értékkészlete
 - $\mathbb{F}$ = az összes lehetséges algoritmust megvalósító függvények halmaza
 - $(\mathbb{F} = \{\mathbb{T}_1 \times \dots \times \mathbb{T}_n \times \mathbb{S} \rightarrow \mathbb{T}_1 \times \dots \times \mathbb{T}_k \times \mathbb{S} \mid n, k \in \mathbb{N}_0\}$, ahol $\mathbb{T}_i$ tetszőleges típusok, $\mathbb{S}$ pedig tetszőleges állapotok halmaza)
- Műveletei
 - függvényhívás



A function (függvény) adattípus

Python VIRTUÁLIS adattípus

- A függvényre eddig mint algoritmusra gondoltunk.
- A Neumann-architektúra szerint az algoritmusok a számítógépben ugyanúgy (bitsorozatként) és ugyanott (memóriában) tárolódnak mint az adatok.
- A függvény azonosító (név) tulajdonképpen az algoritmus implementációját azonosítja.
- Miért ne adhatnánk ezt az algoritmust értékül egy másik azonosítónak (változónak)?

```
>>> def f(x):  
...     return x ** x  
...  
>>> variable = f  
>>> variable(3)  
27  
>>> variable(4)  
256
```